

Algorithm Analysis

When analyzing iterative algorithms, the **time complexity depends on how the loop variable changes**. Nearly every iterative algorithm can be categorized into one of a few common iteration patterns. Mastering these patterns lets you analyze almost any loop.

1. Constant Increment (Linear)

Pattern

```
for(i = 1; i <= n; i++)
```

Or

```
for(i = 0; i < n; i += c)
```

where c is a constant.

Analysis

Number of iterations

```
n/c = O(n)
```

Example

```
for(i=0;i<n;i++)  
{  
    printf("%d",i);  
}
```

Iterations

```
0,1,2,3,...,n-1
```

Time Complexity

$O(n)$

2. Constant Decrement (Linear)

Pattern

```
for(i=n;i>0;i--)
```

Or

```
for(i=n;i>0;i-=5)
```

Iterations

$n, n-1, n-2, \dots$

Time Complexity

$O(n)$

3. Multiplication (Logarithmic)

Pattern

```
for(i=1;i<=n;i*=2)
```

Iterations

1
2
4
8
16
...

Suppose after k iterations,

$$2^k = n$$

Taking log on both sides

$$k = \log n$$

Time Complexity

$$O(\log n)$$

4. Division (Logarithmic)

Pattern

```
for(i=n;i>=1;i/=2)
```

Iterations

n
n/2
n/4
n/8
...
1

Complexity

$$O(\log n)$$

5. Exponential Growth

Pattern

```
for(i=2;i<=n;i=i*i)
```

Iterations

```
2  
4  
16  
256  
...
```

Suppose

$$2^{2^k} = n$$

Taking log twice

$$k = \log \log n$$

Time Complexity

$$O(\log \log n)$$

6. Square Root Increment

Pattern

```
for(i=1;i*i<=n;i++)
```

Since

$$i^2 \leq n$$

Maximum value of i

$$\sqrt{n}$$

Complexity

$O(\sqrt{n})$

7. Cube Root Increment

```
for(i=1;i*i*i<=n;i++)
```

Iterations

$\sqrt[3]{n}$

Complexity

$O(n^{1/3})$

8. Powers

Pattern

```
for(i=1;i<=sqrt(n);i++)
```

Complexity

$O(\sqrt{n})$

9. Variable Increment

Pattern

```
for(i=1;i<=n;i=i+2)
```

Iterations

1
3
5
7
...

Approximately

$n/2$

Complexity

$O(n)$

10. Increasing by i

```
i=1;  
while(i<n)  
    i=i+i;
```

Equivalent to multiplication by 2.

Complexity

$O(\log n)$

11. Increasing by Square

```
i=2;
```

```
while(i<n)
    i=i*i;
```

Complexity

$O(\log \log n)$

12. Triangular Iteration

```
for(i=1;i<=n;i++)
    for(j=1;j<=i;j++)
```

Iterations

```
1
2
3
...
n
```

Total

$$1+2+3+\dots+n = n(n+1)/2$$

Complexity

$O(n^2)$

13. Reverse Triangle

```
for(i=n;i>=1;i--)
    for(j=1;j<=i;j++)
```

Iterations

```
n  
n-1  
n-2  
...  
1
```

Again

$$n(n+1)/2$$

Complexity

$$O(n^2)$$

14. Nested Independent Loops

```
for(i=1;i<=n;i++)  
    for(j=1;j<=m;j++)
```

Iterations

$$n \times m$$

Complexity

$$O(nm)$$

Note: If $m = n$ then $O(n^2)$

15. Triple Nested Loops

```
for(i=1;i<=n;i++)  
  for(j=1;j<=n;j++)  
    for(k=1;k<=n;k++)
```

Iterations

$n \times n \times n$

Complexity

$O(n^3)$

16. Nested Logarithmic

```
for(i=1;i<=n;i++)  
  for(j=1;j<=n;j*=2)
```

Outer loop

n

Inner loop

$\log n$

Complexity

$O(n \log n)$

17. Nested Square Root

```
for(i=1;i<=n;i++)  
    for(j=1;j*j<=n;j++)
```

Outer

n

Inner

$\sqrt{n}$

Complexity

$O(n\sqrt{n})$

18. Log × Log

```
for(i=1;i<=n;i*=2)  
    for(j=1;j<=n;j*=2)
```

Both loops

$\log n$

Complexity

$O((\log n)^2)$

19. Log × Linear

```
for(i=1;i<=n;i*=2)
    for(j=1;j<=n;j++)
```

Complexity

$O(n \log n)$

20. Two Consecutive Loops

```
for(i=1;i<=n;i++)
for(j=1;j<=n;j++)
```

Complexity

$n+n=2n$
 $=O(n)$

21. Three Consecutive Loops

```
for(i=1;i<=n;i++)
for(j=1;j<=n*n;j++)
for(k=1;k<=log n;k++)
```

Complexity

$n + n^2 + \log n$

Dominant Term

$O(n^2)$

22. Geometric Progression

```
for(i=1;i<=n;i*=3)
```

Iterations

1
3
9
27
...

Complexity

$3^k = n$

$k = \log n$

$O(\log n)$

23. Arithmetic Progression

```
for(i=5;i<=n;i+=5)
```

Iterations

5
10
15
...

Approximately

$n/5$

Complexity

$O(n)$

24. Harmonic Iteration

```
for(i=1;i<=n;i++)  
    for(j=1;j<=n/i;j++)
```

Total iterations

$n(1 + 1/2 + 1/3 + 1/4 + \dots + 1/n)$

Using the harmonic series

$H_n = \ln n$

Complexity

$O(n \log n)$

25. Dependent Nested Loop

```
for(i=1;i<=n;i++)  
    for(j=i;j<=n;j++)
```

Iterations

n
n-1
n-2
...
1

Total = $n(n+1)/2$

Complexity

$O(n^2)$

Summary of common iterative patterns

Loop Pattern	Example	Time Complexity
Increment by 1	$i++$	$O(n)$
Decrement by 1	$i--$	$O(n)$
Increment by constant	$i = i + k$	$O(n)$
Multiply by constant	$i = i * 2$	$O(\log n)$
Divide by constant	$i = i / 2$	$O(\log n)$
Square each time	$i = i * i$	$O(\log \log n)$
Square-root bound	$i * i < n$	$O(\sqrt{n})$
Cube-root bound	$i * i * i \leq n$	$O(n^{1/3})$
Two consecutive loops	$n+n$	$O(n)$
Nested linear	$n * n$	$O(n^2)$
Triple nested	n^3	$O(n^3)$
Linear X log	$n \times \log n$	$O(n \log n)$
Log x log	$\log n \times \log n$	$O((\log n)^2)$
Harmonic	$\sum(n/i)$	$O(n \log n)$
Triangular loops	$\sum i$	$O(n^2)$

Reverse Triangular	$\sum(n - i)$	$O(n^2)$
Independent loops	$n \times m$	$O(nm)$
Consecutive mixed loops	$n + n^2 + \log n$	$O(\max(n^2))$

Harmonic Series

The **harmonic series** is one of the most important series in algorithm analysis because it appears in the time complexity of many algorithms, such as nested loops with variable increments, sieve algorithms, and union-find analysis.

The harmonic series is:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n}$$

Although it grows forever, it grows **very slowly**. The key result is

$$H_n = O(\log n)$$

Analysing Recursive Algorithms

When analyzing **recursive algorithms**, we often express the running time using a **recurrence relation**. A recurrence relates the time for a problem of size n to the time for smaller subproblems.

What is a Recurrence Relation?

A recurrence is an equation of the form:

$$T(n) = \text{Time for recursive calls} + \text{Work done outside recursion}$$

where:

$T(n)$ = running time for input size n

Base case gives the stopping condition.

Example:

$$T(1) = 1$$

means a problem of size 1 takes constant time

General Form

$$T(n) = aT(b/n) + f(n)$$

where :

- a = number of recursive calls
- n/b = size of each subproblem
- $f(n)$ = work done outside recursion

Example 1: Recursive Factorial

```
Factorial(n)
  if n == 1
    return 1
  return n × Factorial(n-1)
```

Recurrence

Each call makes **one recursive call**.

$$T(n) = T(n-1) + 1$$

Base Case:

$$T(1) = 1$$

Expansion

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= T(n-2) + 1 + 1 \\ &= T(n-3) + 1 + 1 + 1 \\ &\dots \\ &= T(1) + n - 1 \end{aligned}$$

$$\begin{aligned} \text{Since } T(1) &= 1 \\ T(n) &= n \end{aligned}$$

Therefore

$$T(n) = \Theta(n)$$

Example 2: Recursive Sum of Array

```
Sum(A,n)
  if n <= 0
    return 0

  return A[n - 1] + Sum(A,n-1)
```

Recurrence

$$T(n) = T(n-1) + 1$$

Result

$$\Theta(n)$$

Example 3: Binary Search

```
BinarySearch(A, low, high, key)
if(low > high) return
mid = (low + high) / 2
if(key == Data[mid])
    return mid
else if(key < Data[mid])
    return BinarySearch(A, low, mid - 1, key)
else
    return BinarySearch(A, mid + 1, high, key)
```

Each recursive call searches only half.

Recurrence

$$T(n) = T(n/2) + 1$$

Expansion

```
T(n) = T(n/2) + 1
      = T(n/4) + 2
      = T(n/8) + 3
      ....
Stop when
n/2^k = 1

Therefore
2^k = n

Taking logarithm
k = log n
```

Hence

$$T(n) = 1 + \log n$$

Therefore

$$\Theta(\log n)$$

Example 4: Merge Sort

Merge Sort divides into two equal halves.

Recurrence

$$T(n) = 2T(n/2) + n$$

where

- two recursive calls
- merge takes n

Expansion

Level 0 ----- n

Level 1 ----- $2(n/2) = n$

Level 2 ----- $4(n/4) = n$

Every level costs is n

Number of levels is $\log n$

Total

$$n + n + n + \dots = n \log n$$

Therefore

$$\Theta(n \log n)$$

Example 5: Fibonacci (Naive Recursion)

```
Fib(n)

if n<=1
    return n

return Fib(n-1)+Fib(n-2)
```

Recurrence

$$T(n) = T(n-1) + T(n-2) + 1$$

Base Case

$$T(0) = 0, T(1) = 1$$

Expand the Recurrence

Start with
 $T(n) = T(n-1) + T(n-2) + 1$

Expand $T(n-1)$
 $T(n) = [T(n-2) + T(n-3) + 1] + T(n-2) + 1$
 $= 2T(n-2) + T(n-3) + 2$

Expand one of the $T(n-2)$ terms
 $T(n) = 2[T(n-3) + T(n-4) + 1] + T(n-3) + 2$
 $= 3T(n-3) + 2T(n-4) + 4$

The expression becomes increasingly complicated because each recursive call creates two more recursive calls.

Recursion Tree

```

      F(5)
     /  \
    F(4)  F(3)
   /  \  /  \
  F(3) F(2) F(2) F(1)
 / \  / \  / \
F(2)F(1)F(1)F(0)F(1)F(0)
 / \
F(1) F(0)

```

This generates many repeated computations.

Count number of calls

Let $C(n)$ be the number of function calls
Then

$$C(n) = C(n-1) + C(n-2) + 1$$

The growth follows the same pattern as the Fibonacci sequence.

For small values ,n	Calls
0	1
1	1
2	3
3	5
4	9
5	15
6	25
7	41
8	67

The number of calls increases very rapidly.

Find an Upper Bound

Since

$$T(n-2) \leq T(n-1),$$

we have

$$\begin{aligned} T(n) &= T(n-1) + T(n-2) + 1 \\ &\leq T(n-1) + T(n-1) + 1 \\ &= 2T(n-1) + 1 \end{aligned}$$

Ignoring constants

$$T(n) \leq 2T(n-1)$$

Expand

$$\begin{aligned} T(n) &\leq 2T(n-1) \\ &\leq 2(2T(n-2)) \\ &= 2^2 T(n-2) \\ &= 2^3 T(n-3) \end{aligned}$$

.....

$$= 2^{(n-1)} T(1)$$

Since $T(1)$ is constant

$$T(n) = O(2^n)$$

The solution grows exponentially:

$$T(n) = \Theta(2^n)$$

Example 6: Tower of Hanoi

Move(n)

Move(n-1)

Move largest disk

Move(n-1)

Recurrence

$$T(n) = 2T(n-1) + 1$$

Expand

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2(2T(n-2) + 1) + 1 \\ &= 4T(n-2) + 3 \\ &= 8T(n-3) + 7 \end{aligned}$$

....

Solution

$$T(n) = 2^n - 1$$

Complexity

$$\Theta(2^n)$$

Steps to Analyze a Recursive Algorithm

1. **Identify the base case.**
2. **Write the recurrence relation** by counting:
 - Number of recursive calls.
 - Size of each recursive subproblem.
 - Extra work done outside recursion.
3. **Solve the recurrence** using:
 - Expansion (iteration) method.
 - Substitution method.
 - Recursion tree.
 - Master Theorem (for divide-and-conquer recurrences).
4. **Express the result using Big-O, Big-Ω, or Big-Θ notation.**

Power of a Number Using Recursion

The **Power of a Number using Recursion** is a classic example of a recursive algorithm. It can also be analyzed using a recurrence relation.

Given a base x and an exponent n , compute x^n

using recursion

Example:

- $2^5 = 32$
- $3^4 = 81$

Recursive Idea

$$x^n = x \times x^{n-1}$$

Pseudocode

```
Power(x, n)
if n == 0
    return 1
return x * Power(x, n-1)
```

Recursive Calls

```
Power(2,4)
= 2 × Power(2,3)
Power(2,3)
= 2 × Power(2,2)
Power(2,2)
= 2 × Power(2,1)
Power(2,1)
= 2 × Power(2,0)
Power(2,0)
= 1
```

Returning Back

```
Power(2,1)
=2×1
=2
```

```
Power(2,2)
=2×2
=4
```

```
Power(2,3)
=2×4
=8
```

```
Power(2,4)
=2×8
=16
```

Recursion Tree

```
Power(2,4)
|
Power(2,3)
|
Power(2,2)
|
Power(2,1)
|
Power(2,0)
```

Unlike Fibonacci, **only one recursive call is made at each level**, so the recursion forms a straight chain rather than a branching tree.

Time Complexity Analysis

Step 1: Write the Recurrence

Each call performs:

- One recursive call
- One multiplication (constant time)

Therefore,

$$T(n) = T(n-1) + 1$$

Base case:

$$T(0) = 1$$

Step 2: Expand the Recurrence

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &= T(n-2) + 1 + 1 \\ &= T(n-3) + 1 + 1 + 1 \\ &\dots \\ &= T(0) + n \end{aligned}$$

$$\begin{aligned} \text{Since } T(0) &= 1 \\ T(n) &= n + 1 \end{aligned}$$

Step 3:
Complexity

$$T(n) = \Theta(n)$$

Summary

Problem	Recurrence	Time Complexity
Factorial	$T(n) = T(n-1) + 1$	$\Theta(n)$
Sum of first n numbers	$T(n) = T(n-1) + 1$	$\Theta(n)$
Power of a number	$T(n) = T(n-1) + 1$	$\Theta(n)$
Sum of Array Elements	$T(n) = T(n-1) + 1$	$\Theta(n)$
Find Maximum in an Array	$T(n) = T(n-1) + 1$	$\Theta(n)$
Reverse a String	$T(n) = T(n-1) + 1$	$\Theta(n)$
Check Palindrome	$T(n) = T(n-2) + 1$	$\Theta(n)$
Decimal to Binary	$T(n) = T(n/2) + 1$	$\Theta(\log n)$
Binary Search	$T(n) = T(n/2) + 1$	$\Theta(\log n)$
Merge Sort	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Tower of Hanoi	$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$
Fibonacci (Naive)	$T(n) = T(n-1) + T(n-2) + 1$	$O(2^n)$